

Flask Day 8

Jinja2 Templates & Forms Handling

Student Notes System (SNS) project milestone

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's build

Template variables
Control statements
Loops and filters
Reusable base layout
HTML forms + request.form

Outcome: dynamic forms and reusable templates

Where Day 8 fits in the SNS build

PROJECT CONTEXT



Yesterday

Students created routes, templates, static CSS, base.html, and starter pages.

Today

We make pages dynamic and interactive using Jinja expressions, logic, loops and HTML form submission.

Tomorrow

We connect registration inputs to validation and MySQL persistence.

Learning outcomes

OUTCOMES

Concept

Explain how Flask sends context to a template and how Jinja produces HTML.

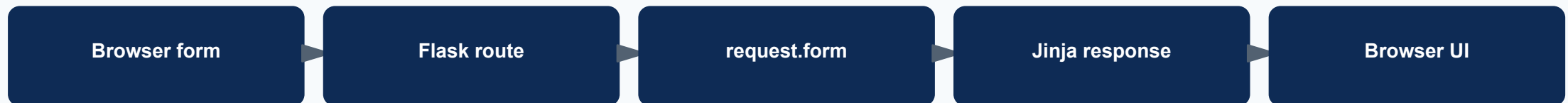
Implementation

Use variables, conditions, loops, filters, inheritance and form controls in SNS pages.

Testing

Test empty input, valid input, missing fields and wrong form method.

By the end of class, students should be able to explain the data direction:



Recap: Day 7 project shell

RECAP

```
student_notes_manager/  
├── app.py  
├── templates/  
│   ├── base.html  
│   ├── home.html  
│   ├── register.html  
│   └── login.html  
└── static/css/app.css
```

Question to class

If the same navbar appears in home.html, login.html and register.html, where should it live?

Expected answer

Move shared HTML into base.html. Child templates should only fill page-specific blocks.

Day 8 starts from this shell and adds dynamic content + form handling.

Core feature flow

FLOW



Context

Python values passed from `render_template()`. Example: `username`, `notes`, `errors`.

Evaluation

Jinja reads variables, runs if/for blocks and applies filters.

Escaping

Untrusted text is escaped before HTML reaches the browser.

Teaching line: Flask decides behaviour. Jinja decides presentation.

Jinja syntax: value vs logic

SYNTAX

{{ variable }}

Displays a value in HTML.
Example: `{{ user.name }}`

{% statement %}

Runs template logic.
Example: `{% if user %} ... {% endif %}`

```
{{ app_name }}  
  
{% if notes %}  
    {{ notes|length }} notes found  
{% endif %}
```

Class check: Which syntax displays? Which syntax controls?

Passing context from Flask to template

CONTEXT

```
@app.get('/dashboard')
def dashboard():
    notes = [
        {'title': 'Flask Routing'},
        {'title': 'Jinja Templates'}
    ]
    return render_template(
        'dashboard.html',
        username='Ravi',
        notes=notes
    )
```

Python variable

Context name

Template variable

Important

A template does not magically know every Python variable. Pass values explicitly using `render_template()`.

Student mistake

Undefined variable usually means the route did not pass the value or the template name is wrong.

Control statements: if / else

CONDITIONS

```
{% if error %}  
  <div class="alert alert-danger">  
    {{ error }}  
  </div>  
{% else %}  
  <p>Fill the form carefully.</p>  
{% endif %}
```

Use cases in SNS

- Show validation errors
- Show logged-in navigation
- Show empty notes message
- Show success message after submit

Rule

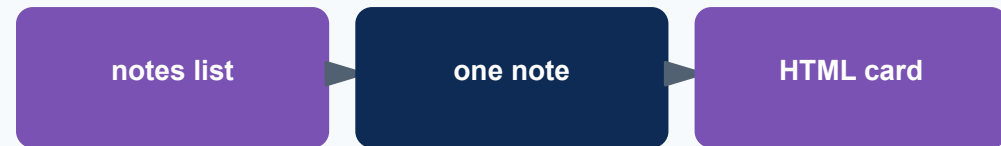
Keep business decisions in Python. Keep display decisions in Jinja.

Loops: displaying a list of notes

LOOPS

```
{% if notes %}
  {% for note in notes %}
    <article class="note-card">
      {{ loop.index }}. {{ note.title|e }}
    </article>
  {% endfor %}
{% else %}
  <p>No notes yet.</p>
{% endif %}
```

Loop mental model



Empty state

Never leave a blank dashboard. Tell the user what to do next: “No notes yet. Add your first note.”

Filters: formatting and safety helpers

FILTERS

|e

Escapes unsafe HTML. Useful when displaying user-controlled text.

|length

Counts items or characters. Useful for note counts.

|default

Shows a fallback when a value is missing.

```
{{ note.title|e }}  
{{ notes|length }} notes  
{{ username|default('Student') }}
```

Security reminder: do not use |safe unless you know exactly why.

Template inheritance: reusable layout

INHERITANCE

base.html

<head> + CSS

navbar

{% block content %}

footer

extends

fills block

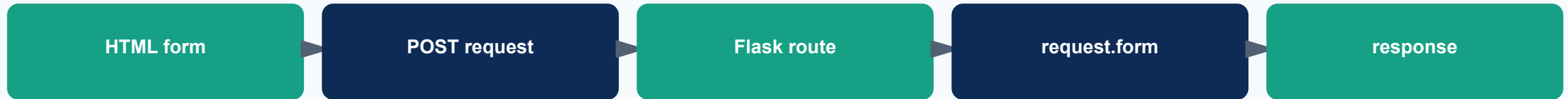
final HTML

```
{% extends 'base.html' %}

{% block content %}
  <h1>Dashboard</h1>
{% endblock %}
```

Forms: the first interactive screen

FORMS



GET

Used to display pages and read public query parameters.

POST

Used to submit data that can change server state.

Validation

Never trust browser inputs. Check required fields before saving.

SNS examples: Register form, Login form, Add Note form.

HTML form structure

HTML

```
<form method="POST" action="/register">
  <input name="name" required>
  <input name="email" type="email" required>
  <input name="password" type="password" required>
  <button type="submit">Register</button>
</form>
```

name attribute

This is the key Flask reads from request.form.

method="POST"

Sends form data in the request body instead of the URL.

required

Browser-level help only. Server-side validation is still required.

Reading form data in Flask

FLASK

```
from flask import request, render_template

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip()
        password = request.form.get('password', '')

        if not name or not email or not password:
            return render_template('register.html',
                                   error='All fields are required')

    return render_template('register.html')
```

Trainer checkpoint

Ask: which values came from the browser? Which line validates them?

Important

Use `.get()` to avoid `KeyError` and `.strip()` to remove accidental spaces.

Next day

Day 9 stores validated users in MySQL.

Validation workflow before saving

VALIDATION



Client-side validation

HTML required/type=email improves user experience.

Server-side validation

Flask must reject bad data even if browser rules are bypassed.

Database constraints

UNIQUE, NOT NULL and FK rules protect data integrity.

Do not commit data changes until validation succeeds.

Form feedback with flash messages

FEEDBACK

```
from flask import flash, redirect, url_for

if not email:
    flash('Email is required', 'danger')
    return redirect(url_for('register'))

flash('Registration form accepted', 'success')
```

```
{% with messages =
get_flashed_messages(with_categories=True) %}
  {% for category, message in messages %}
    <div class="alert alert-{{ category }}">
      {{ message }}
    </div>
  {% endfor %}
{% endwith %}
```

Flash messages are stored for the next response, then displayed once.

SNS pages to update today

MILESTONE

base.html

Add blocks for title/content and a flash-message area.

register.html

Create student registration form with method POST.

login.html

Create login form and reuse the same base layout.

dashboard.html

Display sample notes with if/for and empty state.

README.md

Document new routes, forms and how to test them.

Common errors and coaching response

DEBUGGING

Symptom	Coaching response
Undefined variable	Pass value from render_template or use safe default
Jinja logic too complex	Move business logic into Python
HTML injection risk	Keep autoescape; avoid unnecessary safe filter
Form value missing	Check input name and request.form key

Debug rule: reproduce -> inspect -> identify layer -> change one thing -> retest.

Live coding plan

CLASS PLAN

1. Update base.html

title, content, flash blocks

2. Create register form

method POST + named inputs

3. Handle request.form

read, strip, validate

4. Render errors

show friendly response

5. Dashboard sample notes

if/for, |e, empty state

6. Commit milestone

Day 8: implement Jinja2 templates & forms

Guided lab: Day 8 milestone

GUIDED LAB

Lab goal

Build dynamic register/login/dashboard templates and handle a basic form submission safely.

Independent improvement

Add one improvement: better validation message, empty state, reusable macro/partial, accessibility fix or extra test case.

At least one valid form submission demonstrated

At least two invalid cases demonstrated

Screenshots or screen recording evidence

README updated with routes and test procedure

Clean Git commit with Day 8 message

Quick quiz / exit ticket

CHECK

What is the difference between `{{ }}` and `{% %}`?
Where does Flask look for templates by default?
Why should `base.html` contain the navbar?
How does `request.form` get values from input fields?
Why is browser-side required not enough?
What does `|e` do and why is it important?
What is one failure case you tested today?

Expected answer style

Students must answer with file names, route names, input direction and failure reasoning.

Exit ticket

One concept learned, one error fixed, one commit pushed.

Homework assignment

HOMEWORK

Finish base.html blocks and flash area
Create register/login forms with POST methods
Handle at least one POST route using request.form
Display sample notes with if/for and empty state
Capture valid + invalid submission screenshots
Commit: Day 8: implement jinja2 templates & forms handling

Submission: GitHub link + screenshots + README update

Day 8 recap

RECAP

Jinja variables

Conditions

Loops

Inheritance

Forms

Today we converted static pages into reusable, dynamic and interactive screens.

Remember

Flask owns behaviour. Jinja owns presentation. Browser input is always untrusted.

Next: validated registration stored in MySQL